UNIT 3 · TOPIC 3.1

Network Vulnerabilities and Attacks

Assignment

Instructions: 24 practice questions + 4 coding assignment(s), 41 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

3.1.a · TCP/IP fundamentals and network topology flaws

1. Which TCP/IP layer is responsible for routing packets between networks using IP addresses? (1 pt)

- ○ A. Application layer
- ○ B. Transport layer
- ○ C. Internet layer
- ○ D. Network access layer

2. Which of the following are structural weaknesses built into TCP/IP that attackers exploit? Select ALL that apply. (3 pts)

- □ A. IP addresses are not authenticated
- □ B. DNS caches can accept forged replies
- □ C. TCP uses encrypted sequence numbers by default
- □ D. ARP operates on trust with no authentication

(Select ALL that apply.)

3. An attacker floods a switch with thousands of fake MAC addresses until it begins broadcasting all frames to every port. What attack is this? (1 pt)

- ○ A. MAC flooding
- ○ B. IP spoofing
- ○ C. ARP poisoning
- ○ D. DNS spoofing

4. What network design practice limits how far an attacker can move laterally after compromising a single internal host? (1 pt)

5. During a TCP three-way handshake, what is the correct order of messages? (1 pt)

- ○ A. ACK, SYN, SYN-ACK
- ○ B. SYN, ACK, SYN-ACK
- ○ C. SYN-ACK, SYN, ACK
- ○ D. SYN, SYN-ACK, ACK

6. Which layer of the TCP/IP stack does IP spoofing primarily target? (1 pt)

- ○ A. Application layer
- ○ B. Transport layer
- ○ C. Internet layer
- ○ D. Network access layer

3.1.b · DNS attacks (spoofing, cache poisoning)

7. Why can an attacker forge a DNS response over UDP more easily than they could tamper with a TCP connection? (1 pt)

- ○ A. UDP responses are digitally signed by default
- ○ B. UDP always encrypts responses
- ○ C. UDP requires the attacker to be on the same physical cable
- ○ D. UDP is connectionless, so there is no handshake the resolver can use to verify the reply's origin

8. Which THREE fields must a forged DNS reply match for a resolver to accept it as the answer to an outstanding query? (1 pt)

- ○ A. TTL, checksum, and MAC address
- ○ B. Source IP address only
- ○ C. Destination port, query domain name, transaction ID
- ○ D. HTTP status code, cookie, and referrer

9. Which of the following are effective defenses against DNS cache poisoning? Select ALL that apply. (2 pts)

- □ A. Source port randomization
- □ B. DNSSEC cryptographic signatures
- □ C. Hiding the resolver's IP address
- □ D. Increasing the TTL of all records to one week

(Select ALL that apply.)

10. In the Kaminsky attack, why does the attacker query unique, non-existent subdomains (e.g. 1.bank.com, 2.bank.com) instead of the base domain? (1 pt)

- ○ A. It is required by the DNS protocol specification
- ○ B. It bypasses DNSSEC entirely
- ○ C. It forces the resolver to issue a fresh query to the authoritative server every time, since there is no cached answer, giving the attacker repeated chances to win the race
- ○ D. It reduces the number of packets the attacker needs to send

11. Name the DNS extension that adds cryptographic signatures so a resolver can verify a reply even if an attacker wins the UDP timing race. (1 pt)

12. A resolver's outstanding query has tx_id=500 and source port=9000. A reply arrives with tx_id=500, port=9000, but for a different query name than was asked. What should the resolver do? (1 pt)

- ○ A. Cache it anyway since the tx_id matches
- ○ B. Reject the reply as a mismatch

○ C. Cache it anyway since the port matches

○ D. Forward it to the authoritative server for verification

3.1.c · Man-in-the-middle and ARP spoofing

13. Why does ARP allow spoofing so easily? (1 pt)

○ A. ARP requires a password for every reply

○ B. ARP replies must be digitally signed by the gateway

○ C. ARP only works over encrypted channels

○ D. ARP has no built-in authentication, so any device can send a reply and it will be trusted

14. An attacker wants to intercept all of a victim's internet-bound traffic on a LAN using ARP spoofing. Which two devices' ARP tables does the attacker typically poison? (1 pt)

○ A. Two random unrelated hosts on the network

○ B. The victim's and the default gateway's

○ C. Only the DNS server's

○ D. Only the victim's web browser cache

15. Which of the following are effective technical controls against ARP spoofing? Select ALL that apply. (2 pts)

□ A. Dynamic ARP Inspection (DAI) on switches

□ B. Static ARP entries for critical hosts like the gateway

□ C. Renaming the Wi-Fi network's SSID

□ D. Network segmentation with VLANs

(Select ALL that apply.)

16. Does HTTPS fully prevent damage from a successful ARP-spoofing man-in-the-middle position? (1 pt)

○ A. Yes, HTTPS makes MITM positioning impossible

○ B. No, HTTPS provides no protection at all against MITM

○ C. No, HTTPS protects the confidentiality/integrity of the payload, but the attacker can still see which servers are contacted and can attempt downgrade or certificate-warning attacks

○ D. Yes, once ARP is poisoned, HTTPS automatically re-routes traffic safely

17. Name the switch-level feature that validates ARP packets against a trusted IP-to-MAC binding table and drops forged ones. (1 pt)

18. An ARP-watch tool observes IP 192.168.1.1 first bound to MAC aa:11, then 40 seconds later sees a reply claiming 192.168.1.1 is at MAC bb:22. How should this be classified? (1 pt)

○ A. SPOOFED, a different MAC now claims an IP that was already bound to a different MAC

○ B. LEGIT, ARP bindings change routinely

○ C. Cannot be determined without a packet capture

○ D. LEGIT, only DNS bindings can be spoofed, not ARP

3.1.d · Packet capture and traffic sniffing

19. What NIC setting must a capture tool typically use to see traffic not addressed to its own MAC address? (1 pt)

- ○ A. DHCP mode
- ○ B. Airplane mode
- ○ C. Promiscuous mode
- ○ D. Bridge mode

20. What is the difference between a capture filter and a display filter in a tool like Wireshark? (1 pt)

- ○ A. They are the same thing with different names
- ○ B. A capture filter determines what is recorded at all; a display filter narrows the view of what was already captured
- ○ C. A display filter determines what is recorded; a capture filter only changes colors
- ○ D. Capture filters only work on Windows

21. Which of the following protocols send credentials in plaintext, making them fully readable in a packet capture? Select ALL that apply. (2 pts)

- □ A. Telnet
- □ B. HTTPS
- □ C. FTP (control channel)
- □ D. SSH

(Select ALL that apply.)

22. On a fully switched (non-hub) network, which technique lets an attacker still see another host's traffic without physical access to a SPAN port? (1 pt)

- ○ A. Renaming their hostname
- ○ B. Simply plugging into any switch port
- ○ C. Changing their own IP address
- ○ D. ARP spoofing to redirect the victim's traffic through the attacker's machine

23. Name the legitimate switch feature, intended for monitoring tools, that copies traffic from one or more ports to a monitoring port, and which becomes a risk if granted to the wrong party. (1 pt)

24. A capture log entry shows TCP port 443 traffic to a banking site with payload described only as encrypted binary data. What should an analyst conclude? (1 pt)

- ○ A. No credential can be extracted from this capture because TLS encrypts the payload
- ○ B. The password is visible if you look closely enough
- ○ C. Port 443 is always cleartext
- ○ D. This is definitely a Telnet session

Coding Assignments (Python 3)

3.1.a · TCP/IP fundamentals and network topology flaws

3 pts · auto-graded on the site

Coding problem, 3.1.a: TCP/IP fundamentals and network topology flaws

Skill: Analyze Risk · **Maps to:** 3.1.A · **Language:** Python 3 (pytest)

Problem

A network monitoring tool has captured a log of IP packets on a corporate subnet. Each log entry records the packet's source IP, destination IP, and an expected source subnet (CIDR notation). Your job is to implement a spoofing detector: given a list of packet records, return the zero-based indices of packets whose source IP address falls outside the expected source subnet, which is a strong indicator of IP spoofing.

A packet is considered suspicious (spoofed) when `src_ip` does not belong to `expected_subnet`. Only IPv4 addresses are present. Subnets are given as `"A.B.C.D/N"` CIDR strings.

Function contract

Implement `solve(packets: list[dict]) -> list[int]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of dicts. Each dict has three string keys:

- `"src_ip"`, e.g. `"10.0.1.55"`
- `"dst_ip"`, e.g. `"10.0.1.1"`
- `"expected_subnet"`, e.g. `"10.0.1.0/24"`

Output

A sorted list of zero-based integer indices of the suspicious (spoofed) packets. Empty list if none are suspicious.

Examples

```
[{"src_ip": "10.0.1.55",  "dst_ip": "10.0.1.1", "expected_subnet": "10.0.1.0/24"},
 {"src_ip": "192.168.5.9","dst_ip": "10.0.1.1", "expected_subnet": "10.0.1.0/24"},
 {"src_ip": "10.0.1.200", "dst_ip": "10.0.1.1", "expected_subnet": "10.0.1.0/24"}]
-> [1]          # packet at index 1 has src_ip outside 10.0.1.0/24

[]  ->  []      # empty log: nothing to flag
```

Constraints

- Deterministic, offline, standard library only (`ipaddress` module is standard library).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
from typing import Any

def solve(data: Any) -> list[int]:
    """
    Identifies spoofed packets where the source IP is outside the expected subnet.

    Args:
        data: A list of dictionaries, each containing keys: "src_ip", "dst_ip", and "expected_subnet".
    Returns:
```

```
        A sorted list of integer indices of packets where the source IP is not in the expected subnet.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

3.1.b · DNS attacks (spoofing, cache poisoning)

3 pts · auto-graded on the site

Coding problem, 3.1.b: DNS resolver race classifier

Skill: Analyze Risk, Detect Attacks · **Maps to:** 3.1.A, 3.1.B · **Language:** Python 3 (pytest)

Problem

You are building triage logic for a resolver reply log at Crestfield Analytics. For each outstanding query, the log records the replies that arrived, and, for each reply, who sent it (`"authoritative"` or `"attacker"`), and the `tx_id`, `port`, and `qname` it carried. Classify each query's outcome:

- **`"REJECT"`**, no reply is a valid candidate (none match the query's `tx_id`, `port`, AND `qname` all three), so nothing is cached. This also covers the case of zero replies.
- **`"WIN"`**, among the valid candidates (replies matching all three fields), the one with the smallest `arrival` number (i.e. the first to arrive) has `"from": "authoritative"`.
- **`"POISON"`**, among the valid candidates, the first-arriving one has `"from": "attacker"`.

Function contract

Implement `solve(queries)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `queries`: `list[dict]`, each with: - `"qname"`: `str`, the domain name queried - `"tx_id"`: `int`, the transaction ID the resolver sent - `"port"`: `int`, the source port the resolver sent the query from - `"replies"`: `list[dict]`, each with `"from"` (`"authoritative"` or `"attacker"`), `"tx_id"` (`int`), `"port"` (`int`), `"qname"` (`str`), `"arrival"` (`int`, lower = earlier)

Output

A `list[str]` of classifications, **one per input query, in the same order as the input**. Each entry is one of `"WIN"`, `"POISON"`, `"REJECT"`.

Examples

```
queries = [
    {
        "qname": "bank.example", "tx_id": 48291, "port": 33412,
        "replies": [
            {"from": "attacker", "tx_id": 48291, "port": 33412, "qname": "bank.example", "arrival": 1},
            {"from": "authoritative", "tx_id": 48291, "port": 33412, "qname": "bank.example", "arrival": 2},
        ],
    },
]
solve(queries)  ->  ["POISON"]

queries2 = [
    {
        "qname": "shop.example", "tx_id": 11020, "port": 40500,
        "replies": [
            {"from": "attacker", "tx_id": 99999, "port": 40500, "qname": "shop.example", "arrival": 1},
```

```
            {"from": "authoritative", "tx_id": 11020, "port": 40500, "qname": "shop.example", "arrival": 2},
        ],
    },
]
solve(queries2)  ->  ["WIN"]
```

Constraints

- Deterministic, offline, standard library only.
- A reply is a "valid candidate" only if `reply["tx_id"] == query["tx_id"]` AND `reply["port"] == query["port"]` AND `reply["qname"] == query["qname"]`.
- Among valid candidates, only the one with the minimum `arrival` value matters (test data never has ties on the minimum `arrival`).
- Replies may be given in any order in the list; use the `arrival` field, not list position, to determine which arrived first.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.1.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List, Dict


def solve(queries: List[Dict]) -> List[str]:
    """Classify each DNS query's resolver race outcome as WIN, POISON, or REJECT.

    Args:
        queries: list of dicts with keys 'qname' (str), 'tx_id' (int), 'port' (int),
                 'replies' (list of dicts with 'from', 'tx_id', 'port', 'qname', 'arrival').
    Returns:
        List of classification strings, same length and order as `queries`, each one of
        "WIN", "POISON", "REJECT".
        See prompt.md and interface.md for the exact rules.
    """
    # TODO: implement the classification rules per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

3.1.c · Man-in-the-middle and ARP spoofing

3 pts · auto-graded on the site

Coding problem, 3.1.c: ARP spoofing detector

Skill: Analyze Risk · **Maps to:** 3.1.B · **Language:** Python 3 (pytest)

Problem

You are building an arpwatch-style detector for Crestfield Analytics. Given a timestamp-ordered log of observed ARP replies (each mapping an IP to a MAC), identify which IP addresses show the ARP spoofing signature: the **first** reply seen for an IP establishes its trusted MAC binding. Any **later** reply for the same IP with a **different** MAC is a spoofing indicator. A later reply with the *same* MAC as the trusted binding is just a normal refresh and is not flagged.

Function contract

Implement `solve(replies)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `replies`: `list[dict]`, each with: - `"ts"`: `int`, Unix timestamp (seconds) the reply was observed - `"ip"`: `str`, the IP address claimed in the reply - `"mac"`: `str`, the MAC address claimed for that IP

Output

A `list[str]`, the distinct IP addresses that show the spoofing signature (at least one reply with a MAC different from that IP's first-seen MAC), **sorted alphabetically**, with no duplicates.

Examples

```
replies = [
    {"ts": 100, "ip": "192.168.1.1", "mac": "aa:11:22:33:44:01"},
    {"ts": 130, "ip": "192.168.1.1", "mac": "aa:11:22:33:44:01"},
    {"ts": 145, "ip": "192.168.1.1", "mac": "de:ad:be:ef:00:99"},
]
solve(replies)  ->  ["192.168.1.1"]

replies2 = [
    {"ts": 100, "ip": "192.168.1.50", "mac": "11:22:33:44:55:66"},
]
solve(replies2)  ->  []
```

Constraints

- Deterministic, offline, standard library only.
- `replies` may be given in any timestamp order in the list; use the `ts` field (not list position) to determine which reply for a given IP came first (the "trusted" binding).
- Test data never has two replies for the same IP with an identical minimum `ts`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.1.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List, Dict


def solve(replies: List[Dict]) -> List[str]:
    """Flag IP addresses that show the ARP spoofing signature.

    Args:
        replies: list of dicts with keys 'ts' (int), 'ip' (str), 'mac' (str).
    Returns:
        Sorted list (no duplicates) of IP addresses where a later reply's MAC differs from
        the MAC of the earliest (by 'ts') reply for that IP.
        See prompt.md and interface.md for the exact rules.
    """
    # TODO: implement the detection rule per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

3.1.d · Packet capture and traffic sniffing

3 pts · auto-graded on the site

Coding problem, 3.1.d: Packet capture credential-leak detector

Skill: Detect Attacks · **Maps to:** 3.1.B, 3.1.C · **Language:** Python 3 (pytest)

Problem

You are triaging a captured packet log at Crestfield Analytics. For each entry, determine whether it exposes a plaintext credential ("LEAK") or not ("SAFE"):

- **Encrypted ports are always "SAFE"**, regardless of payload content: port 443 (HTTPS) and port 22 (SSH) encrypt their payload, so nothing in the payload text can be treated as an actual leaked credential.
- **Cleartext-capable ports** are 80 (HTTP), 23 (Telnet), and 21 (FTP). An entry on one of these ports is **"LEAK"** if its payload contains any credential indicator string (case-sensitive substring match), otherwise it is **"SAFE"**.
- Credential indicator strings to check for (substring match anywhere in the payload): `"password="`, `"Password:"`, `"PASS "`.
- Any other port not listed above is treated as `"SAFE"` (out of scope for this detector).

Function contract

Implement `solve(packets)` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `packets`: `list[dict]`, each with: - `"port"`: `int`, destination port - `"payload"`: `str`, the captured payload text

Output

A `list[str]` of classifications, **one per input packet, in the same order as the input**. Each entry is one of `"LEAK"`, `"SAFE"`.

Examples

```
packets = [
    {"port": 80, "payload": "POST /login HTTP/1.1\r\n\r\nuser=alice&password=Pass123!"},
    {"port": 443, "payload": "\\x16\\x03\\x03 encrypted application data"},
]
solve(packets)  ->  ["LEAK", "SAFE"]

packets2 = [
    {"port": 23, "payload": "login: admin\r\nPassword: R0uterAdmin!"},
    {"port": 80, "payload": "GET /index.html HTTP/1.1"},
]
solve(packets2)  ->  ["LEAK", "SAFE"]
```

Constraints

- Deterministic, offline, standard library only.
- Port check first: if `port` is 443 or 22, the result is always `"SAFE"`.
- Otherwise, if `port` is 80, 23, or 21: check for any of `"password="`, `"Password:"`, `"PASS "` as a substring of `payload`; if found, `"LEAK"`, else `"SAFE"`.
- Any other port: always `"SAFE"`.

- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 3.1.d. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List, Dict


def solve(packets: List[Dict]) -> List[str]:
    """Classify each packet as LEAK (cleartext credential exposed) or SAFE.

    Args:
        packets: list of dicts with keys 'port' (int) and 'payload' (str).
    Returns:
        List of classification strings, same length and order as `packets`, each one of
        "LEAK", "SAFE".
        See prompt.md and interface.md for the exact rules.
    """
    # TODO: implement the classification rules per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```